

MODULE 15 · DOMAIN 4 - OPERATIONAL EFFICIENCY AND OPTIMIZATION

Watching It Run

Task 4.3. Bedrock metrics, quality signals, tracing, anomaly detection and dashboards.

DOMAIN 4 of 5 EXAM WEIGHT 12% MODULE 15 of 22

Traditional monitoring watches for errors. A generative AI system's worst failures produce **no errors at all**: a 200 response, normal latency, and an answer that is wrong.

Four layers, and most teams watch one

01

Infrastructure

Lambda, ECS, EKS. CPU, memory, container health.

02

Application

API latency, error rates, request volume.

03

Model

Tokens, invocation latency, throttles, guardrails.

04

Quality

Is the answer right?
Nothing publishes this.

ON THE EXAM

Holistic observability means all four. A team watching only the first two has a dashboard that stays green through every failure that matters in this course.

The metrics you get without doing anything

METRIC	ANSWERS
<code>Invocations</code>	How much are we calling the model?
<code>InvocationLatency</code>	Time to the final token. Total latency
<code>TimeToFirstToken</code>	The streaming experience metric
<code>InputTokenCount, OutputTokenCount</code>	Where is the token spend going?
<code>InvocationThrottles</code>	Are we exceeding RPM or TPM?
<code>InvocationClientErrors, InvocationServerErrors</code>	Whose fault is the failure?

ON THE EXAM

Learn the split between the two latency metrics. `InvocationLatency` is what a machine waits for; `TimeToFirstToken` is what a human experiences. Alarming on the wrong one hides the complaint.

Metrics or invocation logs?

- ◆ **Metrics** are numeric, near real-time and cheap. They tell you *that* something changed.
- ◆ **Invocation logs** hold the actual prompts and responses. They tell you *what* changed.
- ◆ **Logs Insights** queries them, and results pin to a dashboard as a log widget.
- ◆ Attribution by application or tenant needs the logs. Metric dimensions are coarse.
- ◆ Logs are off by default, cost real money at volume, and carry a privacy obligation.

ON THE EXAM

"Real-time alerting on thresholds" means metrics. "Analyse usage patterns and attribute them" means Logs Insights over invocation logs. Many questions want both, deliberately.

A company runs several Bedrock applications across business units. Some show unpredictable token consumption. It needs real-time visibility into token usage across multiple models, custom dashboards for different stakeholder groups, and alerting when consumption crosses thresholds.

The company also wants to attribute usage to individual applications to identify which ones drive the spikes. Operational overhead must be as low as possible.

Which two solutions together meet the requirements? (Select TWO.)

- ☐ A Route invocation events through EventBridge and Data Firehose into OpenSearch Serverless, and analyse with OpenSearch dashboards
- ☐ B Use CloudWatch Logs Insights over Bedrock invocation logs to analyse consumption and attribute it by application, adding log widgets to dashboards
- ☐ C Build CloudWatch dashboards from the native Bedrock token and invocation metrics, with alarms on token usage thresholds
- ☐ D Configure a zero-ETL integration to Amazon Managed Grafana for token usage trends

Which two solutions together meet the requirements? (Select TWO.)

- A** EventBridge, Firehose, OpenSearch

A working pipeline you build and operate yourself. Three services and an index to maintain for data CloudWatch already holds.
- B** Logs Insights over invocation logs, log widgets

Native metric dimensions are coarse, so per-application attribution comes from the logs. Log widgets put those queries on the same dashboards.
- C** CloudWatch dashboards on native metrics plus alarms

Bedrock publishes token and invocation metrics already, so real-time visibility and threshold alerting need no pipeline at all.
- D** Zero-ETL to Managed Grafana

Grafana is a legitimate dashboard surface, and this adds a service to reach data CloudWatch already exposes. Higher overhead for the same view.

ON THE EXAM

Multiple-response questions are all-or-nothing, so both halves must be right. C covers real-time alerting; B covers attribution. Neither alone answers the whole question.

No AWS metric tells you whether the answers are **good**. Quality monitoring is something you construct, and skill 4.3.2 names it explicitly.

Measuring quality in production

- ◆ **Sample and judge.** Score a share of live responses with an evaluation model, emit it as a custom metric.
- ◆ **Guardrail interventions.** A rising block rate is a quality signal, not only a safety one.
- ◆ **Empty or low-score retrievals.** The best early warning that a RAG system is drifting.
- ◆ **User behaviour.** Thumbs, escalation, question repetition, abandonment. Free and meaningful.
- ◆ **Run the golden set on a schedule** and publish the pass rate as a metric.

ON THE EXAM

Escalation rate is the most underrated signal here. When users stop trusting an assistant they do not complain; they route around it, and that shows up as human contacts rising.

An internal assistant has been in production for eight months. Every CloudWatch dashboard is green: latency stable, no throttling, error rate near zero, token usage flat. Nothing has been deployed for six weeks.

Support ticket volume has nonetheless risen 30% over two months, and the helpdesk reports staff saying the assistant "used to be better". The team cannot reproduce a failure and has no data showing anything wrong.

What should the team put in place?

- ☐ A Increase CloudWatch metric resolution and add more infrastructure alarms
- ☐ B Run the golden set on a schedule and publish pass rate as a metric, sample live responses for LLM-as-a-judge scoring, and track empty-retrieval and escalation rates
- ☐ C Enable AWS X-Ray tracing to find where latency is being introduced
- ☐ D Redeploy the most recent known-good version of the application

◆ PAUSE AND DECIDE

What should the team put in place?

- A

 More resolution, more infrastructure alarms
More precision on metrics that are already correct. The infrastructure is genuinely healthy; the answers are not.
- B

 Golden set on a schedule, sampled judging, retrieval and escalation rates
Builds the quality layer the team never had. Rising escalation is the ticket increase explained, and the corpus is the likely drifter.
- C

 X-Ray tracing
Excellent for finding where time goes. Nobody has reported a latency problem, and the metrics confirm there is not one.
- D

 Redeploy a known-good version
Nothing has been deployed for six weeks, so there is no regression to roll back to. The code is not what changed.

ON THE EXAM

"Nothing deployed for six weeks" plus rising complaints is the drift signature from module 13. The corpus, the users or the questions moved, and only quality metrics would have shown it.

Guardrail metrics are operational signals too

- ◆ **Intervention counts** tell you how often a guardrail actually blocked or masked something.
- ◆ Metrics break down by **policy type and by source**, so you can see whether input or output fired.
- ◆ **A sudden rise** means an attack, a content change, or a policy that is now too tight.
- ◆ **A sudden fall to zero** usually means the guardrail stopped being attached, not that behaviour improved.
- ◆ Alarm on **both directions**. Module 11's monitor mode is how you set the baseline before enforcing.

ON THE EXAM

The fall-to-zero case is the one nobody alarms on. A deployment that drops the guardrail parameter looks like a wonderful improvement on a dashboard.

Tracing across the FM boundary

- ◆ **AWS X-Ray** follows a request across API Gateway, Lambda, retrieval and the model call.
- ◆ The value is **attribution of time**. Three seconds is retrieval, model, or your own code.
- ◆ **FM interaction tracing** is the guide's phrase: which prompt, which tools, which steps.
- ◆ Propagate a **correlation ID** so a user complaint maps to exactly one trace.
- ◆ **Business impact metrics with custom dashboards** is also named in the skill.

ON THE EXAM

Retrieval latency is the component most often unmeasured. A RAG request can spend a third of its time in the vector store before the model is even invoked.

CloudWatch generative AI observability

- ◆ **GA since October 2025.** Latency, token usage, errors and performance, out of the box.
- ◆ Covers **AgentCore Runtime, Built-in Tools, Gateway, Memory and Identity** as one view.
- ◆ Compatible with **Strands Agents, LangChain and LangGraph**. Not tied to one framework.
- ◆ Gives **end-to-end prompt tracing**, curated metrics and logs, in an AgentCore console section.
- ◆ Integrates with **Application Signals, alarms and Logs Insights**.

ON THE EXAM

This is the answer when a scenario wants agent-level visibility without building it. Instrumenting an agent by hand is the option that fails on operational overhead.

Container Insights or GenAI observability?

CONTAINER INSIGHTS

The infrastructure layer

- CPU, memory, pod and task health
- ECS and EKS cluster performance
- Tells you the container is healthy
- Blind to what the model returned

GENAI OBSERVABILITY

The model and agent layer

- Token usage, prompt traces, tool calls
- Agent step-level visibility
- Tells you what the model did
- Blind to node-level resource pressure

THE TIEBREAKER

A scenario about FM behaviour deviating from baselines is not an infrastructure question. Container Insights appears in option sets precisely because it sounds thorough.

An agent built on AgentCore with six tools takes between 4 and 40 seconds to respond, apparently at random. The team cannot tell whether the variation comes from the number of tool calls, a slow tool, retrieval, or the model itself.

They need step-level visibility into what the agent did on each run, correlated with the user's request, and want to avoid building custom instrumentation.

What should the team enable?

- A CloudWatch Container Insights on the compute running the agent
- B CloudWatch generative AI observability, giving connected traces across AgentCore runtime, tools, gateway and memory
- C Model invocation logging, and analyse prompt lengths with Logs Insights
- D Custom CloudWatch metrics emitted from each tool with the embedded metric format

◆ PAUSE AND DECIDE

What should the team enable?

- A

 Container Insights
Infrastructure health for compute that is largely idle while waiting on tool calls. It cannot see a single agent step.
- B

 GenAI observability across AgentCore components
Purpose-built for exactly this: step-level traces across runtime, tools, gateway and memory, with no instrumentation to write.
- C

 Invocation logging plus Logs Insights
Shows the model calls and not the tool executions between them. Variable agent latency is usually the loop, not one prompt.
- D

 Custom metrics from each tool
Would work, and it is the build-it-yourself answer the scenario rules out. It also misses the orchestration time between tools.

ON THE EXAM

"4 to 40 seconds at random" across an agent loop is a variable iteration count, so you need step-level tracing. "Avoid building custom instrumentation" then eliminates D.

Watching the tools, not just the model

- ◆ **Per-tool invocation count.** A tool never chosen is a description problem, as module 7 argued.
- ◆ **Per-tool error rate and latency.** One slow tool caps every run that touches it.
- ◆ **Tool selection accuracy.** Sample runs and check whether the chosen tool was right.
- ◆ **Iteration count per session.** A rising average means more work for the same outcome.
- ◆ **Stopping-condition hits.** Every run hitting the cap is a run that failed to finish.

ON THE EXAM

Iteration count is the agent metric to alarm on. It rises quietly when a tool starts returning worse data, long before any error rate moves.

Anomaly detection, three kinds

- ◆ **Token burst patterns.** CloudWatch anomaly detection learns the normal band and flags departures.
- ◆ **Response drift.** Response length, refusal rate or judge score moving with no deployment.
- ◆ **Cost anomaly detection** at the billing layer, catching what no application metric would.
- ◆ Anomaly detection beats static thresholds when normal is **seasonal**. A Monday peak is not an incident.
- ◆ Static thresholds still win where a hard limit exists, such as approaching a quota.

ON THE EXAM

The exam guide names token burst patterns and response drift specifically. Both are cases where a fixed threshold either fires constantly or never fires at all.

Three surfaces, and a rename to know

SURFACE	BEST FOR	NOTE
CloudWatch dashboards	Engineers, operational metrics, alarms	Native. Lowest overhead
Amazon Managed Grafana	Teams standardised on Grafana	Multi-source. Another service to run
Amazon Quick	Business stakeholders, executives	Formerly QuickSight, then Quick Suite

ON THE EXAM

QuickSight became Quick Suite in October 2025, then simply Quick. QuickSight now names the BI component inside it. The in-scope list says Amazon Quick; older material says QuickSight.

A platform team already publishes Bedrock operational metrics to CloudWatch dashboards and is happy with them. The finance director and three business unit heads now want a monthly view of AI cost and usage per business unit, alongside existing revenue reporting they already consume.

The finance team does not use the AWS console and will not be given access to it. Engineering wants to add as little operational overhead as possible.

What should the platform team do?

- A Give the finance team read-only CloudWatch access and share the existing dashboards
- B Surface Cost and Usage Report data, split by application inference profile tags, in Amazon Quick alongside the existing business reporting
- C Stand up Amazon Managed Grafana and rebuild the operational dashboards there for finance
- D Export CloudWatch metrics to Amazon S3 monthly and email a spreadsheet

◆ PAUSE AND DECIDE

What should the platform team do?

- A

 Read-only CloudWatch access
The scenario states finance will not be given console access. It also shows technical metrics to an audience that wants cost per business unit.
- B

 CUR by profile tags, surfaced in Amazon Quick
Cost data at the right granularity, on the BI surface finance already uses, next to the revenue reporting that gives it meaning.
- C

 Rebuild in Managed Grafana
Another service to run, still an engineering tool, rebuilding dashboards that already work for the audience that already has them.
- D

 Monthly spreadsheet export
A manual process somebody must remember, with no drill-down and no link to the revenue reporting alongside it.

ON THE EXAM

Two audiences want two surfaces. Engineers keep CloudWatch; finance gets a BI tool. The application inference profile tags from module 14 are what make the cost split possible at all.

Alarms that are worth waking someone for

- ◆ **Every alarm names an action.** If nobody knows what to do, it is a dashboard widget, not an alarm.
- ◆ **Throttles and server errors** page. Latency and token spend usually do not.
- ◆ **Alarm on rates and ratios**, not raw counts. Errors rise with traffic; error *rate* does not.
- ◆ **Composite alarms** stop one incident producing nine pages from nine correlated metrics.
- ◆ **Quality alarms need patience.** A judge score is noisy, so alarm on a trend rather than one bad sample.

ON THE EXAM

The rate-versus-count point matters most on a growing product. An alarm on absolute error count fires every time you succeed at getting more users.

What observability costs you

- ◆ **Invocation logs are large.** Full prompts and responses at volume can rival the model bill.
- ◆ **Set log retention deliberately.** The default is forever, and forever is expensive.
- ◆ **Logs Insights charges per gigabyte scanned,** so an unbounded dashboard query recurs.
- ◆ **Sample rather than log everything** when you only need a statistical picture.
- ◆ **Alarms nobody acts on are worse than none.** Each one should name an action.

ON THE EXAM

Log volume is the observability bill people are surprised by. S3 with lifecycle rules is cheaper than CloudWatch Logs for long-retention invocation data you rarely query.

A team enabled Bedrock model invocation logging to CloudWatch Logs across all environments to support a compliance requirement. Six weeks later, CloudWatch Logs is the second-largest line on the AWS bill, ahead of Bedrock itself.

The compliance requirement applies only to the production customer-facing application, and requires seven years of retention. Development and test environments were included because it was simpler to enable everywhere.

What should the team do?

- A Disable model invocation logging and rely on CloudTrail for the compliance requirement
- B Log only the production application, deliver those logs to Amazon S3 with lifecycle tiering for the seven-year retention, and set a short retention on any CloudWatch Logs kept for operational queries
- C Reduce the CloudWatch Logs retention period to 30 days across all environments
- D Keep logging everywhere and compress the log data before ingestion

What should the team do?

- A

Disable logging, use CloudTrail
Module 13's distinction. CloudTrail records that a call happened and holds no prompt or response, so the compliance requirement fails.
- B

Production only, S3 with lifecycle, short CloudWatch retention
Scopes logging to where it is required, puts seven-year retention on the storage class built for it, and keeps a short operational window.
- C

30-day retention everywhere
Cuts the bill and breaks the seven-year requirement, which is the reason logging was enabled in the first place.
- D

Compress the logs
Reduces volume somewhat while still storing dev and test content nobody needs, in the most expensive place, forever.

ON THE EXAM

Two dimensions were over-applied: scope and destination. It was enabled everywhere when only production needed it, and sent to CloudWatch Logs when S3 is the seven-year home.

The whole domain on one slide

- ◆ **Everything is tokens.** Send fewer, generate fewer, or avoid the invocation entirely.
- ◆ **Application inference profiles** attribute spend. Daily aggregate, not per request, never retroactive.
- ◆ **Three latency metrics.** TTFT for humans, InvocationLatency for machines, throughput for peaks.
- ◆ **Metrics say *that*. Logs say *what*.** Native metrics for alarms, Logs Insights for attribution.
- ◆ **Green dashboards prove nothing about quality.** That layer is one you build.

ON THE EXAM

That is 12% of the exam, and 89% now covered. Module 16 opens Domain 5 with evaluation: how you decide whether any of this is actually working.